Ex. No.	4	**Name**	Ajmal Rahman A
Date	18-10-2026	**Reg. No.**	192324094
Sub. Name	Deep Learning	**Sub. Code**	CSA4702
Semester	MS2 September 2026		

EXERCISE NAME

CATEGORICAL CROSS-ENTROPY FOR IMAGE CLASSIFICATION

SCENARIO

An image-classification model predicts a probability for each of several object categories. Calculate the categorical cross-entropy between the true class labels and the predicted probabilities for a batch of images, and show how the loss of an image depends on the probability that the model assigns to its true category.

ALGORITHM

Step 1 : Import the packages used in this exercise — NumPy for the array algebra, SciPy for the softmax function and the entropy function, and Matplotlib for the plots.

Step 2 : Fix the five object categories — Cat, Dog, Car, Bird and Aeroplane — and the true category of each of the 10 images in the batch.

Step 3 : Encode the true labels as a one-hot matrix Y of order 10 × 5, in which $y_{ik} = 1$ if image i belongs to category k and $y_{ik} = 0$ otherwise, so that every row of Y is a probability distribution concentrated on a single category.

Step 4 : Take the raw scores produced by the output layer of the network for the 10 images as a matrix of logits of order 10 × 5.

Step 5 : Convert every row of logits into a probability distribution with the softmax function, $p_{ik} = e^{(z_{ik})} / \Sigma_j e^{(z_{ij})}$, subtracting the largest logit of the row before exponentiating so that the computation stays numerically stable. Verify that every row of the probability matrix P sums to one.

Step 6 : Compute the categorical cross-entropy of every image as $L_i = -\Sigma_k y_{ik} \log p_{ik}$. Because the label is one-hot, every term of the sum except the true category is multiplied by zero, so the loss reduces to −log p of the true category alone.

Step 7 : Compute the loss of the batch as the mean of the ten individual losses, and the classification accuracy from the category of the largest predicted probability of every row.

Step 8 : Verify the result in two independent ways — the batch loss must equal −mean(log p of the true class), and, since the entropy of a one-hot label is zero, the cross-entropy must equal the Kullback–Leibler divergence $D(Y \| P)$ from the true distribution to the predicted one.

Step 9 : Tabulate the loss obtained when the probability given to the true category is 0.99, 0.90, 0.70, 0.50, 0.30, 0.10 and 0.01, to show that the penalty grows without bound as the model becomes confident in a wrong category.

Step 10 : Plot the loss of every image in the batch, separating the correctly classified images from the misclassified ones.

Step 11 : Plot the loss curve L = −log p and mark the ten images on it at the probability each one received for its true category. Stop.

SOURCE CODE

Module 1 : Importing the Required Packages

```
import numpy as np
from scipy.special import softmax
from scipy.stats import entropy
import matplotlib.pyplot as plt

plt.rcParams["font.family"]          = "serif"
plt.rcParams["font.serif"]           = ["Cambria", "Caladea", "DejaVu Serif"]
plt.rcParams["axes.spines.top"]      = False
plt.rcParams["axes.spines.right"]    = False
plt.rcParams["legend.frameon"]       = False
```

Module 2 : Defining the Object Categories and the True Labels

```
np.random.seed(42)
CLASSES = ["Cat", "Dog", "Car", "Bird", "Aeroplane"]
K       = len(CLASSES)                            # number of object categories
M       = 10                                     # number of images in the batch

y_true  = np.array([0, 2, 1, 4, 3, 2, 0, 3, 1, 4])        # true class index
Y       = np.eye(K)[y_true]                           # one-hot matrix , (10, 5)

print("Categories :", CLASSES)
print("True class index of every image :", y_true)
print("One-hot label matrix Y :", Y.shape)
print(Y.astype(int))
```

Module 3 : Converting the Network Scores into Probabilities

```
logits = np.array([
    [ 4.2,  1.1, -1.0,  0.6, -0.8],   # confident and correct
    [-0.9,  0.2,  3.8, -1.1,  0.4],   # confident and correct
    [ 1.6,  2.0, -0.7,  0.3, -0.5],   # correct but not confident
    [-0.6,  0.1, -0.2,  1.0,  2.9],   # confident and correct
    [ 0.2, -0.4,  0.5,  2.6,  0.9],   # confident and correct
    [ 1.9, -0.3,  1.7,  0.2, -0.6],   # wrong : Cat preferred over Car
    [ 2.8,  1.4, -0.9,  0.1, -0.7],   # confident and correct
    [ 0.4,  0.9,  0.3,  1.1,  0.7],   # uncertain but correct
    [ 2.4, -0.2,  0.5,  0.8, -0.3],   # wrong : Cat preferred over Dog
    [-0.5,  0.3,  0.7,  1.2,  3.4]])  # confident and correct

P = softmax(logits, axis=1)                       # predicted probability matrix
print("Probability matrix P :", P.shape)
print("Row sums of P :", np.round(P.sum(axis=1), 10))
print(np.round(P, 4))
```

Module 4 : Softmax Computation for a Single Image

```
z   = logits[0]
ez  = np.exp(z - z.max())                     # shifted for numerical stability
p0  = ez / ez.sum()
print("Image 1 logits        :", np.round(z, 3))
print("Image 1 exponentials  :", np.round(ez, 6))
print("Image 1 probabilities :", np.round(p0, 6), " sum :", p0.sum())
```

Module 5 : Categorical Cross-Entropy of every Image

```
EPS   = 1e-12
P_cl  = np.clip(P, EPS, 1.0)
```

```
losses = -np.sum(Y * np.log(P_cl), axis=1)        # L_i = - sum_k y_ik log p_ik

p_true = P[np.arange(M), y_true]           # probability given to the true class
y_pred = P.argmax(axis=1)                         # predicted class index

for i in range(M):
    print("Image %2d | true %-9s | predicted %-9s | p %.4f | loss %.4f | %s" % (
        i + 1, CLASSES[y_true[i]], CLASSES[y_pred[i]], p_true[i], losses[i],
        "correct" if y_pred[i] == y_true[i] else "wrong"))
```

Module 6 : Batch Loss, Accuracy and Verification

```
batch_loss = losses.mean()              # categorical cross-entropy of the batch
accuracy   = np.mean(y_pred == y_true)

print("Total cross-entropy of the batch : %.6f" % losses.sum())
print("Mean categorical cross-entropy   : %.6f" % batch_loss)
print("Classification accuracy          : %.2f %%" % (100 * accuracy))

simple_check = -np.mean(np.log(p_true))       # same as the mean loss
kl_check     = np.mean([entropy(Y[i], P_cl[i]) for i in range(M)])
print("Loss from -log(p_true) only : %.6f" % simple_check)
print("Mean KL divergence  D(Y||P) : %.6f" % kl_check)
print("Entropy of a one-hot label  : %.6f" % entropy(Y[0]))
```

Module 7 : Effect of the Predicted Confidence on the Loss

```
conf = np.array([0.99, 0.90, 0.70, 0.50, 0.30, 0.10, 0.01])
conf_loss = -np.log(conf)
for c, l in zip(conf, conf_loss):
    print("p(true class) = %.2f -> loss = %.4f" % (c, l))
```

Module 8 : Visualisation of the Loss

```
fig, ax = plt.subplots(figsize=(6.6, 3.3))
ok = y_pred == y_true
ax.bar(np.arange(1, M + 1)[ok],  losses[ok],  color="#2F6DB5",
       label="Correctly classified")
ax.bar(np.arange(1, M + 1)[~ok], losses[~ok], color="#C0702A",
       label="Misclassified")
ax.axhline(batch_loss, color="#4E8F59", lw=1.3, ls="--")
ax.text(0.7, batch_loss + 0.08, "batch mean = %.4f" % batch_loss,
        fontsize=8, color="#4E8F59")
ax.set_xlabel("Image number in the batch")
ax.set_ylabel("Categorical cross-entropy loss")
ax.set_xticks(np.arange(1, M + 1))
ax.grid(axis="y", alpha=0.55);  ax.set_axisbelow(True)
ax.legend(loc="upper left", fontsize=8)
fig.tight_layout()
fig.savefig("plot1_per_image_loss.png", dpi=300, bbox_inches="tight")

fig, ax = plt.subplots(figsize=(6.6, 3.3))
grid = np.linspace(0.01, 1.0, 400)
ax.plot(grid, -np.log(grid), color="#2F6DB5", lw=1.8,
        label="Loss curve  L = $-\\log\\,p$")
ax.scatter(p_true[ok],  losses[ok],  s=30, c="#4E8F59", zorder=3,
           label="Correctly classified images")
ax.scatter(p_true[~ok], losses[~ok], s=30, c="#C0702A", zorder=3,
           label="Misclassified images")
ax.set_xlabel("Probability assigned to the true category,  p")
ax.set_ylabel("Cross-entropy loss of the image")
ax.set_xlim(0, 1.02);  ax.set_ylim(0, 4.8)
ax.set_xticks(np.arange(0, 1.01, 0.1))
ax.grid(alpha=0.55);  ax.set_axisbelow(True)
ax.legend(loc="upper right", fontsize=8)
fig.tight_layout()
```

```
fig.savefig("plot2_loss_curve.png", dpi=300, bbox_inches="tight")
```

OUTPUT

Table 1 - Softmax Conversion for Image 1 (Logits → Probabilities)

S.No	Category	Logit z_k	$\exp(z_k - z_{max})$	Probability p_k	One-hot y_k
1	Cat	+4.20	1.000000	0.921976	1
2	Dog	+1.10	0.045049	0.041534	0
3	Car	-1.00	0.005517	0.005086	0
4	Bird	+0.60	0.027324	0.025192	0
5	Aeroplane	-0.80	0.006738	0.006212	0
—	Total	—	1.084627	1.000000	1

Table 2 - Predicted Probability Matrix P of the Batch (order 10 × 5)

S.No	Image	Cat	Dog	Car	Bird	Aeroplane	True Category
1	IMG-01	0.9220	0.0415	0.0051	0.0252	0.0062	Cat
2	IMG-02	0.0084	0.0254	0.9283	0.0069	0.0310	Car
3	IMG-03	0.3348	0.4994	0.0336	0.0912	0.0410	Dog
4	IMG-04	0.0235	0.0473	0.0350	0.1163	0.7778	Aeroplane
5	IMG-05	0.0628	0.0344	0.0847	0.6917	0.1264	Bird
6	IMG-06	0.4557	0.0505	0.3731	0.0833	0.0374	Car
7	IMG-07	0.7306	0.1802	0.0181	0.0491	0.0221	Cat
8	IMG-08	0.1446	0.2384	0.1308	0.2911	0.1951	Bird
9	IMG-09	0.6698	0.0497	0.1002	0.1352	0.0450	Dog
10	IMG-10	0.0163	0.0362	0.0541	0.0891	0.8043	Aeroplane

Table 3 - Categorical Cross-Entropy of every Image

S.No	Image	True Category	Predicted Category	p(true class)	Loss $-\log p$	Outcome
1	IMG-01	Cat	Cat	0.9220	0.0812	Correct
2	IMG-02	Car	Car	0.9283	0.0744	Correct
3	IMG-03	Dog	Dog	0.4994	0.6943	Correct
4	IMG-04	Aeroplane	Aeroplane	0.7778	0.2512	Correct
5	IMG-05	Bird	Bird	0.6917	0.3686	Correct
6	IMG-06	Car	Cat	0.3731	0.9859	Wrong
7	IMG-07	Cat	Cat	0.7306	0.3139	Correct
8	IMG-08	Bird	Bird	0.2911	1.2340	Correct
9	IMG-09	Dog	Cat	0.0497	3.0008	Wrong
10	IMG-10	Aeroplane	Aeroplane	0.8043	0.2178	Correct
—	Batch total	—	—	—	7.2220	—

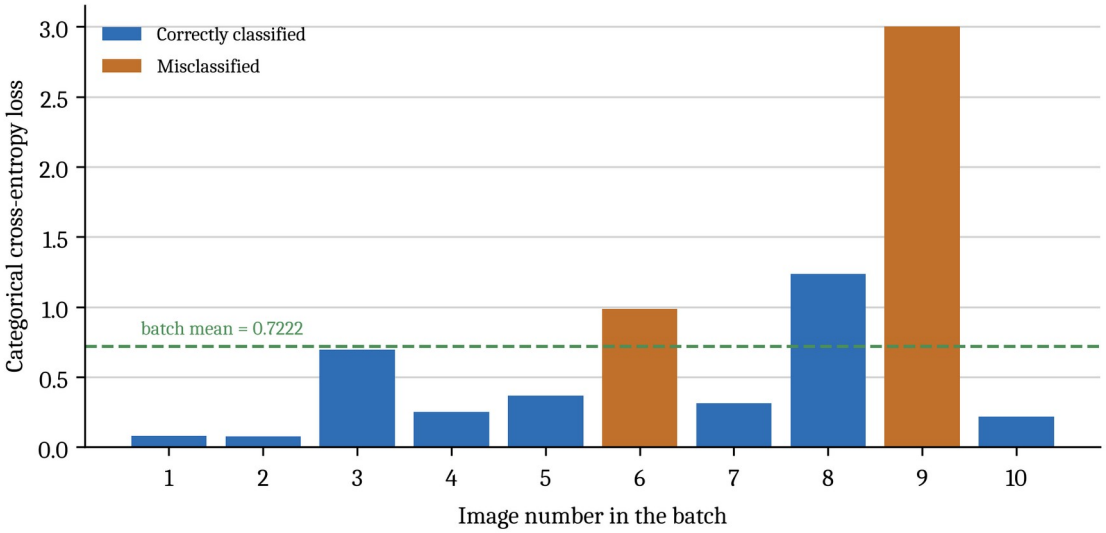
Table 4 - Loss Summary and Verification of the Batch

Parameter	Value
Number of images in the batch (m)	10
Number of object categories (K)	5
Order of the probability matrix P	10 × 5
Order of the one-hot label matrix Y	10 × 5
Row sums of P	1.000000 for every image
Total cross-entropy ΣL_i	7.222001
Mean categorical cross-entropy	0.722200
Same value from −mean(log p_true)	0.722200
Mean KL divergence $D(Y \| P)$	0.722200
Entropy of a one-hot label H(Y)	0.000000
Identity verified	$CE = H(Y) + D(Y \| P) = D(Y \| P)$
Classification accuracy	80.00 %
Lowest loss (best prediction)	IMG-02 , 0.0744
Highest loss (worst prediction)	IMG-09 , 3.0008

Table 5 - Effect of the Predicted Confidence on the Loss

S.No	Probability given to the True Category	Loss $-\log p$	Interpretation
1	0.99	0.0101	Almost no penalty
2	0.90	0.1054	Almost no penalty
3	0.70	0.3567	Small penalty
4	0.50	0.6931	Moderate penalty
5	0.30	1.2040	Moderate penalty
6	0.10	2.3026	Heavy penalty
7	0.01	4.6052	Heavy penalty

Plot 1 - Categorical Cross-Entropy Loss of every Image in the Batch



Plot 2 - Loss Curve −log p with the Ten Images Marked on it

